

Lecture 13: Writing Functions

Harris Coding Camp – Standard Track

Summer 2024

Functions

Example of a function:

```
circle_area <- function(r) {  
  pi * r^2  
}
```

- ▶ Why do we want to write functions?
- ▶ How do we write functions in practice?

Motivation

- ▶ Grolemond and Wickham chapter 19:
 - ▶ “You should consider writing a function whenever you’ve copied and pasted a block of code more than twice.”
 - ▶ This reduces redundancy and makes your code more efficient
- ▶ Some functions have not been defined in R
 - ▶ we need to define it by ourselves

Function

Functions are blocks of reusable code that perform specific tasks or calculations. Functions generally follow three steps:

1. take in **input(s)** (i.e. **argument(s)**)
2. **process** the input(s) according to the code within the function
3. **return** an output, which may be a vector, plot, etc

Example: The `mean()` function:

1. **Inputs:** takes in a set of elements
2. **Process:** calculates the mean of the elements
3. **Return:** a numeric value (i.e. the mean of the input elements)

```
mean(c(1, 2, 3))
```

```
## [1] 2
```

Function anatomy

In R, you define a function using the `function` keyword. The anatomy of a function is as follows:

```
function_name <- function(arg_1, arg_2) {  
  do_this(arg_1, arg_2)  
}
```

Three main components of a function:

1. **Name**

- ▶ The name of your newly defined function

2. **Arguments**

- ▶ These are the inputs your function will take
 - ▶ can be vectors, data frames, etc.
- ▶ In this example, the function takes two inputs: `arg_1`, `arg_2`
- ▶ You need to specify values to these inputs

3. **Body**

- ▶ What your function does to the inputs (`do_this`)

Function anatomy: example

- ▶ **Function name:** circle_area
- ▶ **Argument:** r
- ▶ **Body:** $\pi * r^2$

```
circle_area <- function(r) {  
  pi * r^2  
}
```

```
circle_area(10)
```

```
## [1] 314.1593
```

```
circle_area(c(3, 5, 7))
```

```
## [1] 28.27433 78.53982 153.93804
```

Writing a function: printing output

You start writing code to say Hello to all of your friends.

- ▶ You notice it's getting repetitive. ... time for a function!

```
print("Hello Regina!")
```

```
## [1] "Hello Regina!"
```

```
print("Hello Sheng-Hao!")
```

```
## [1] "Hello Sheng-Hao!"
```

```
print("Hello Jeff!")
```

```
## [1] "Hello Jeff!"
```

Writing a function: start with body & arguments

Start with the **body**:

```
print("Hello Regina!")  
print("Hello Sheng-Hao!")  
print("Hello Jeff!")
```

Q: What part of the code is changing?

- ▶ Your friends' names!
- ▶ Make this an **argument (input)**

Writing a function: define the argument

Rewrite the code to accommodate the **argument**:

```
friend_name <- "Regina"  
cat("Hello", friend_name, "!")
```

```
## Hello Regina !
```

Writing a function: add the structure

Next, let's add the **structure** to formally define this new function:

```
function(friend_name) {  
  cat("Hello", friend_name, "!")  
}
```

- ▶ **argument:** friend_name
- ▶ **body:** cat("Hello", friend_name, "!")
- ▶ What is the **name** of this function?

Writing a function: assign to a name

Try to use names that actively tell the user what the code does.

- ▶ assign to **name** of this function: say_hello_to

```
say_hello_to <- function(friend_name) {  
  cat("Hello", friend_name, "!")  
}
```

First example: printing output

Test out different inputs!

```
say_hello_to("Jasmin")
```

```
## Hello Jasmin !
```

```
say_hello_to("Joan")
```

```
## Hello Joan !
```

```
say_hello_to("Andrew")
```

```
## Hello Andrew !
```

```
# This function also works with vectorized input!
```

```
say_hello_to(c("Jasmin", "Joan", "Andrew"))
```

```
## Hello Jasmin Joan Andrew !
```

Second example: calculating the mean of a sample

Your stats prof asks you to simulate a central limit theorem¹, by calculating the mean of samples from the std normal distribution with increasing sample sizes.

```
rmnorm(1)
```

```
rmnorm(5)
```

```
rmnorm(30)
```

```
mean(rnorm(1))
```

```
## [1] -0.8285424
```

```
mean(rnorm(5))
```

```
## [1] -0.4397587
```

```
mean(rnorm(30))
```

```
## [1] 0.1427961
```

¹Don't worry about this now – we'll come back to this in Fall.

Second example: calculating the mean of a sample

The sample size `n` is changing, meaning it should be the **argument**:

```
calc_sample_mean <- function(n) {  
  mean(rnorm(n))  
}
```

- ▶ **argument**: `n`
- ▶ **body**: `mean(rnorm(n))`
- ▶ assign to **name** of the function: `calc_sample_mean`

```
calc_sample_mean(1)  
calc_sample_mean(5)  
calc_sample_mean(30)
```

Adding additional arguments

If we want to be able to adjust the details of how our function runs, we can add **arguments**:

```
# function which takes 3 arguments
calc_sample_mean <- function(n, our_mean, our_sd) {
  sample <- rnorm(n, mean = our_mean, sd = our_sd)
  mean(sample)
}
```

- ▶ **arguments:** n, our_mean, our_sd
- ▶ **body:**
 - ▶ sample <- rnorm(n, mean = our_mean, sd = our_sd)
 - ▶ mean(sample)

Setting defaults

We can even set **default values** for some arguments:

```
calc_sample_mean <- function(n, our_mean = 0, our_sd = 1) {  
  sample <- rnorm(n, mean = our_mean, sd = our_sd)  
  mean(sample)  
}
```

```
# uses the defaults  
calc_sample_mean(10)
```

```
## [1] 0.05630706
```


Setting defaults

We can change one or two defaults in this example:

```
calc_sample_mean(10, our_sd = 2)
```

```
## [1] -0.7485365
```

```
calc_sample_mean(10, our_mean = 6)
```

```
## [1] 5.884173
```

We can refer by argument name, or use the position:

```
calc_sample_mean(10, 8, 5)
```

```
## [1] 7.970498
```

Setting defaults

This won't work though:

- ▶ the most important argument is missing!

```
calc_sample_mean(our_mean = 5)
```

```
Error in rnorm(n, mean = our_mean, sd = our_sd) :  
  argument "n" is missing, with no default
```

Try it yourself

Adapted from Ben Skinner's *programming 1* R Workshop ([link here](#))

Before presenting task, we'll create a sample dataset `df` that contains some negative values.

First, just run the following chunk of code to create `df`; don't worry about understanding this code.

```
set.seed(54321) # so that we all get the same random numbers
df <- tibble('id' = 1:100,
             'age' = sample(c(seq(11, 20, 1), -97, -98, -99),
                           size = 100,
                           replace = TRUE,
                           prob = c(rep(.09, 10), .1, .1, .1)),
             'sibage' = sample(c(seq(5, 12, 1), -97, -98, -99),
                               size = 100,
                               replace = TRUE,
                               prob = c(rep(.115, 8), .1, .1, .1)),
             'parage' = sample(c(seq(45, 55, 1), -4, -7, -8),
                               size = 100,
                               replace = TRUE,
                               prob = c(rep(.085, 11), .1, .1, .1))
            )
```

Try it yourself

Some common tasks when working with survey data:

- ▶ identify number of observations with **negative** values for a variable
- ▶ identify number of observations with **missing** values for a variable

Your task:

- ▶ Write a function that counts the number of observations with **negative** values for age from `df`. (Hint: use `sum(x < 0)`)
- ▶ Write a function that counts the number of observations with **missing** values for age from `df`. (Hint: use `sum(is.na(x))`)

Conditional execution

`if...else` statements allow you to conditionally execute certain blocks of code depending on whether some condition is satisfied.

```
if (condition is TRUE) {  
    do this  
} else {  
    do this other thing  
}
```

Conditional execution, example

Write a function that takes `gpa` as an input and does the following:

- ▶ if `gpa` is less than 2, function prints GPA and says they are on probation;
- ▶ otherwise, function prints GPA and says they are not on probation.

```
notice_gpa <- function(gpa) {  
  if (gpa < 2) {  
    cat("Your GPA is", gpa, "and you are on academic probation.")  
  } else {  
    cat("Your GPA is", gpa, "and you are not on academic probation.")  
  }  
}
```

```
notice_gpa(1.9)
```

```
## Your GPA is 1.9 and you are on academic probation.
```

```
notice_gpa(3)
```

```
## Your GPA is 3 and you are not on academic probation.
```

Multiple conditions

Use `if...else if...else` statements when you have two or more conditions you want to specify in the code:

```
if (1st condition is TRUE) {  
    do this first thing  
}  
else if (2nd condition is TRUE) {  
    do this second thing  
}  
else {  
    do this other thing  
}
```

Multiple conditions, example

Write a function that takes `gpa` as an input and does the following:

- ▶ if `gpa` less than 2, prints: "Your GPA is `gpa`. You are on academic probation."
- ▶ else if `gpa` is at least 3.5, prints: "Your GPA is `gpa`. You made the Dean's list. Congrats!"
- ▶ otherwise, prints: "Your GPA is `gpa`."

```
notice_gpa_new <- function(gpa) {  
  if (gpa < 2) {  
    cat("Your GPA is", gpa, ". You are on academic probation.")  
  } else if (gpa >= 3.5) {  
    cat("Your GPA is", gpa, ". You made the Dean's list. Congrats!")  
  } else {  
    cat("Your GPA is", gpa, ".")  
  }  
}
```

```
notice_gpa(1.9)  
notice_gpa(3.8)  
notice_gpa(3)
```


Try it yourself

1. Write a function (with arguments a, b, c) which prints out the result of $a * b + c$. What is the result if $a = 11$, $b = 5$, $c = 3$?
2. What will happen if we execute the code below? Any error?

```
# Create a function with arguments  
new.function <- function(a, b) {  
  print(a^2)  
  print(a)  
  print(b)  
}  
  
new.function(6)
```

Try it yourself

You are given a chunk of code which defines a new function:

```
input_1 <- 20
mySum <- function(input_1, input_2 = 10) {
  output <- input_1 + input_2
  return(output)
}
```

3. Given the above code, which value does `mySum(1, 3)` produce?
4. If `mySum(3)` returns 13, why does `mySum(input_2 = 3)` return an error?

Try it yourself

5. (Optional; challenging) Write your own function to compute the **sample variance** of a numeric vector. Sample variance is defined as:

$$s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2$$

Hint: x is a numeric vector and includes more than one element. x_i denotes the i^{th} element in x . For example, $x = c(4, 8, 6, 5, 3)$, $x_4 = 5$.

Recap

- ▶ Write functions when you are using a set of operations repeatedly
- ▶ Functions consist of arguments and a body and are usually assigned to a name
- ▶ Functions are for humans
 - ▶ pick names for the function and arguments that are clear and consistent
- ▶ Debug your code as much as you can as you write it.

For more: See Chapter 26 in R for Data Science (2e)

Appendix

Always test your code

Try to foresee the kind of input you expect to use.

```
calc_sample_mean(1)
```

```
## [1] 0.4382481
```

```
calc_sample_mean(5)
```

```
## [1] -0.3250498
```

We see below that this function is not vectorized. We might hope to get 3 sample means out but only get 1:

```
calc_sample_mean(c(1, 5, 30))
```

```
## [1] -0.143562
```

Why?²

²read ?rnorm to understand how rnorm interprets vector input.

How to deal with unvectorized functions

If we don't want to change our function, but we want to use it to deal with **vectors**, then we have a couple options. Here we use the function `rowwise()`:

```
# create a vector to test our function
sample_tibble <- tibble(sample_sizes = c(1, 5, 30))

# rowwise groups the data by row, allowing calc_sample_mean
# to be applied to each row in sample_tibble
sample_tibble %>%
  rowwise() %>%
  mutate(sample_means = calc_sample_mean(sample_sizes))
```

```
## # A tibble: 3 x 2
## # Rowwise:
##   sample_sizes sample_means
##         <dbl>         <dbl>
## 1             1         -0.833
## 2             5        -0.00741
## 3            30        -0.162
```

Functions in functions

We can pass **functions as arguments** to other functions.

Before:

```
calc_sample_mean <- function(n, our_mean = 0, our_sd = 1) {  
  sample_mean <- mean(rnorm(n, mean = our_mean, sd = our_sd))  
  sample_mean  
}
```

After:

```
summarize_sample <- function(n, our_mean = 0, our_sd = 1, summary_func)  
  summary_stat <- summary_func(rnorm(n, mean = our_mean, sd = our_sd))  
  summary_stat  
}
```


Functions in functions

This function can only calculate sample mean:

```
calc_sample_mean(n = 10, our_mean = 0, our_sd = 1)
```

This function can summarize sample in multiple ways:

```
summarize_sample(n = 10, our_mean = 0, our_sd = 1, summary_func = mean)
summarize_sample(n = 20, our_mean = 5, our_sd = 5, summary_func = sd)
summarize_sample(n = 30, our_mean = 8, our_sd = 3, summary_func = max)
```

Probability distributions

R has built-in functions for working with distributions.

Category	Function	Operation
r	<code>rnorm(n)</code>	generates a random sample of size n
p	<code>pnorm(q)</code>	returns CDF value at q
q	<code>qnorm(p)</code>	returns the quantile for a given probability
d	<code>dnorm(x)</code>	returns pdf value at x

Probability distributions you are familiar with are likely built-in to R.

For example, the binomial distribution has `dbinom()`, `pbinom()`, `qbinom()`, `rbinom()`. The t distribution has `dt()`, `pt()`, `qt()`, `rt()`, etc.

Read this tutorial for more examples.

•

What is r?

- `rnorm()`: random sampling

```
rnorm(1)
```

```
## [1] 0.4153822
```

```
rnorm(5)
```

```
## [1] 0.48019231 -0.06176704 0.39341229 -0.52347657 0.86224112
```

```
rnorm(30)
```

```
## [1] -0.46491942 0.08225974 1.23387437 -1.43763673 0.60650982 0.  
## [7] 0.56148646 -0.57174024 0.21009923 0.76195475 1.23826609 1.  
## [13] 0.97204983 0.91268608 -0.39102111 -0.55656099 -0.49310361 -0.  
## [19] 0.74710766 0.47677986 0.22428500 -0.66792548 -0.13980387 1.  
## [25] 0.30714468 1.11894442 -0.50293380 0.44824647 -0.09282990 0.
```

What is d?

- ▶ `dnorm()`: density function, the PDF evaluated at X .

```
dnorm(0)
```

```
## [1] 0.3989423
```

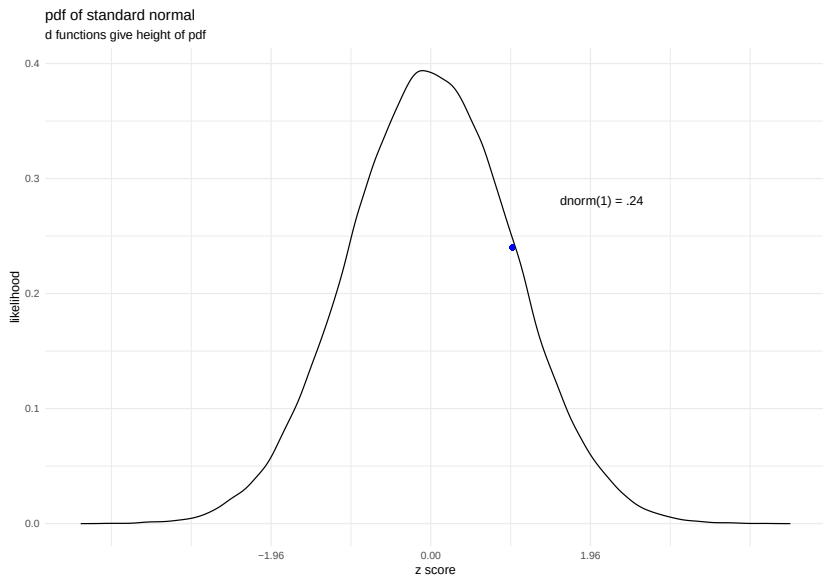
```
dnorm(1)
```

```
## [1] 0.2419707
```

```
dnorm(-1)
```

```
## [1] 0.2419707
```

What is d?



What are p and q?

`pnorm(q)` returns the probability we observe a value less than or equal to some value q .

```
pnorm(1.96)
```

```
## [1] 0.9750021
```

```
pnorm(0)
```

```
## [1] 0.5
```

`qnorm(p)` returns the inverse of `pnorm`. Plug in the probability and get the cutoff.

```
qnorm(.975)
```

```
## [1] 1.959964
```

```
qnorm(.5)
```

```
## [1] 0
```

What are p and q?

